

Reasoning Over Time: State Estimation

Lecture 21

CS 4880

Kirk Duran

Last Time: The World Was Frozen

Bayesian networks let the robot reason about uncertainty in a single snapshot:

$$P(X_1, \dots, X_n) = \prod_i P(X_i \mid \text{Parents}(X_i)).$$

But a delivery robot does not live in a snapshot.

- ▶ the robot moves;
- ▶ pedestrians move;
- ▶ wheel odometry drifts;
- ▶ sensor readings arrive continuously;
- ▶ yesterday's best estimate may be wrong now.

Key idea

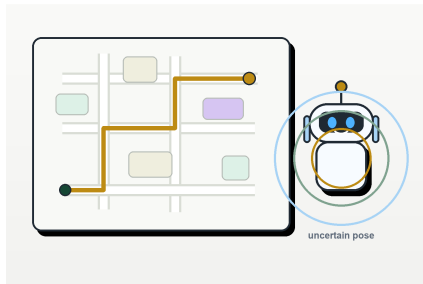
Temporal reasoning asks how an agent should update uncertain beliefs as the hidden world changes over time.

The Campus Robot Now Has a New Problem

The route planner says where the robot *should* be.
The robot still needs to estimate where it *actually* is.

At time t , the robot has:

- ▶ a previous belief about its state;
- ▶ a control command or odometry estimate;
- ▶ a new noisy sensor observation;
- ▶ uncertainty in both motion and sensing.

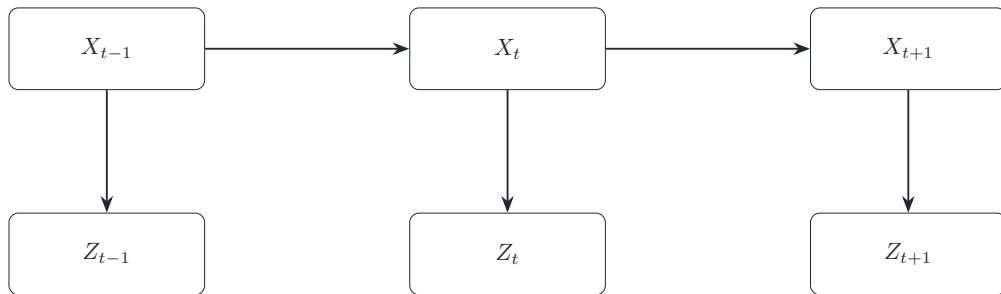


Key idea

Planning chooses a route; state estimation determines where the robot believes it is while executing that route.

A Dynamic Bayesian Network Adds Time

Duplicate the relevant random variables across time slices.



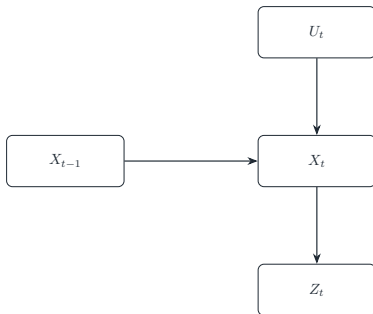
X_t is the hidden state; Z_t is the observation at time t .

Key idea

A dynamic Bayesian network is a Bayesian network whose structure is repeated across time.

For a Robot, Actions Also Affect the Next State

A useful robotics slice often contains a control input U_t :



$P(X_t | X_{t-1}, U_t)$ motion / transition model

$P(Z_t | X_t)$ sensor / observation model

Key idea

State estimation combines what motion predicts with what the sensors observe.

The Temporal Joint Distribution

Under a first-order transition model and local observation model,

$$P(x_{0:T}, z_{1:T} \mid u_{1:T}) = P(x_0) \prod_{t=1}^T P(x_t \mid x_{t-1}, u_t) P(z_t \mid x_t).$$

Every new time step adds:

- ▶ one new hidden state;
- ▶ one new observation;
- ▶ one transition factor;
- ▶ one observation factor.

Key idea

The model remains locally simple even as the complete trajectory becomes very large.

Temporal Models Support Different Questions

Task	Query	Question
Filtering	$P(X_t \mid z_{1:t})$	Where am I now?
Prediction	$P(X_{t+k} \mid z_{1:t})$	Where might I be later?
Smoothing	$P(X_k \mid z_{1:t}), k < t$	Where was I then?
Most likely sequence	$\arg \max x_{1:t} P(x_{1:t} \mid z_{1:t})$	What path most likely occurred?

Today we focus on **filtering** / **state estimation**.

Key idea

Filtering is online inference: estimate the current hidden state from all evidence observed so far.

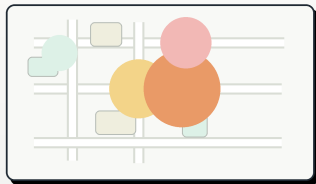
Filtering Maintains a Belief State

Define the robot's belief at time t :

$$bel_t(x) = P(X_t = x \mid z_{1:t}, u_{1:t}).$$

This is not one guessed location.

It is a probability distribution over possible current states.



probability mass over locations

Key idea

State estimation should represent uncertainty, not merely output a single point estimate.

Filtering Is a Two-Step Recursion

1. Predict using motion:

$$\overline{bel}_t(x_t) = \sum_{x_{t-1}} P(x_t | x_{t-1}, u_t) bel_{t-1}(x_{t-1}).$$

2. Correct using the new observation:

$$bel_t(x_t) = \eta P(z_t | x_t) \overline{bel}_t(x_t).$$

η normalizes the distribution.

Key idea

Every filter cycles through the same idea: predict from dynamics, then correct with evidence.

Worked Example: A Three-Zone Campus Corridor

The robot is hidden in one of three zones:

$$X_t \in \{A, B, C\}.$$

It commands `move east`.

The sensor reports whether it sees a distinctive mural near zone C .



Key idea

We will compute one exact filtering update by hand, then examine what happens as the state space and history grow.

Transition Model: Motion Is Not Deterministic

For command `move east`:

	$X_t = A$	$X_t = B$	$X_t = C$
$X_{t-1} = A$	0.20	0.80	0.00
$X_{t-1} = B$	0.00	0.20	0.80
$X_{t-1} = C$	0.00	0.00	1.00

The robot may fail to move because of wheel slip, a pedestrian, or control error.

Key idea

A transition model converts control uncertainty into a distribution over possible next states.

Observation Model: The Mural Sensor Is Noisy

Let $Z_t = +$ mean “mural detected.”

State	$P(Z_t = + \mid X_t)$	$P(Z_t = - \mid X_t)$
A	0.05	0.95
B	0.20	0.80
C	0.90	0.10

The sensor is useful, but neither perfect nor deterministic.

Key idea

The observation model tells us how plausible each sensor reading would be from each hidden state.

Start with an Uncertain Prior

Before moving,

$$bel_0 = [0.60, 0.30, 0.10].$$

So the robot currently believes:

$$P(A) = 0.60, \quad P(B) = 0.30, \quad P(C) = 0.10.$$

It now commands `move east`.

Key idea

Filtering never starts from certainty unless the system genuinely has certainty.

Prediction Step: Push the Belief Through Motion

$$\overline{bel}_1(A) = 0.60(0.20) = 0.12$$

$$\overline{bel}_1(B) = 0.60(0.80) + 0.30(0.20) = 0.54$$

$$\overline{bel}_1(C) = 0.30(0.80) + 0.10(1.00) = 0.34$$

Therefore,

$$\overline{bel}_1 = [0.12, 0.54, 0.34].$$

Key idea

Motion spreads probability mass because uncertain actions create several plausible next states.

Correction Step: The Sensor Reports “Mural”

Suppose $Z_1 = +$.

Unnormalized posterior:

$$\begin{aligned} &[0.12(0.05), 0.54(0.20), 0.34(0.90)] \\ &= [0.006, 0.108, 0.306]. \end{aligned}$$

Normalize by 0.420:

$$bel_1 \approx [0.014, 0.257, 0.729].$$

Key idea

Motion favored zone B , but the new sensor evidence shifts most probability mass to zone C .

That Was Easy Because the State Was Tiny

We tracked only three possible states.

A real robot pose might include

$$X_t = (x_t, y_t, \theta_t, v_t, \text{slip}_t, \text{sensorHealth}_t, \dots).$$

Even if every variable is discretized, the joint state space multiplies:

$$|X| = |x| |y| |\theta| |v| |\text{slip}| \dots$$

Key idea

Exact filtering cost depends on the size and structure of the hidden state representation.

If We Track Full Histories, the Number of Paths Explodes

With n possible states per time step, a length- T state history has

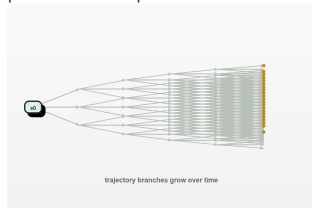
$$n^{T+1}$$

possible trajectories.

Example with only 20 possible states and 10 transitions:

$$20^{11} = 2.048 \times 10^{14}$$

possible state sequences.



Key idea

Explicitly retaining every possible history is not an online inference strategy.

Do We Actually Care About the Entire History?

To estimate the robot's *current* state, do we need to remember

$$X_0, X_1, \dots, X_{t-2}, X_{t-1}$$

individually?

Or can the current state summarize the past information that matters for predicting the future?

This is the modeling decision that changes everything.

Key idea

Tractability comes from deciding what information the present state must preserve about the past.

The First-Order Markov Assumption

Assume the present state contains all information needed to predict the next state:

$$P(X_t \mid X_{0:t-1}, U_{1:t}) = P(X_t \mid X_{t-1}, U_t).$$

The past still matters.

It matters *through the current state distribution*.

Key idea

Markov does not mean “history is irrelevant”; it means the chosen state is a sufficient summary of relevant history for one-step prediction.

Observations Get a Similar Locality Assumption

We usually assume the current observation depends only on the current hidden state:

$$P(Z_t \mid X_{0:t}, Z_{1:t-1}) = P(Z_t \mid X_t).$$

Once X_t is known, earlier states and observations do not directly improve prediction of Z_t .

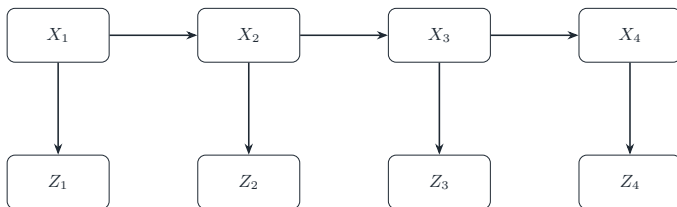
Key idea

Local conditional independence is what lets temporal models reuse the same small transition and observation models at every step.

A Hidden Markov Model Is a Very Simple DBN

A standard HMM has:

- ▶ one hidden state variable X_t per time step;
- ▶ one observation variable Z_t per time step;
- ▶ a first-order Markov transition model;
- ▶ a local observation model;
- ▶ usually stationary parameters reused through time.



Key idea

HMMs are not a different universe from Bayesian networks; they are a restricted temporal Bayesian-network structure.

The HMM Filtering Recursion

Prediction:

$$P(X_t \mid z_{1:t-1}) = \sum_{x_{t-1}} P(X_t \mid x_{t-1})P(x_{t-1} \mid z_{1:t-1}).$$

Correction:

$$P(X_t \mid z_{1:t}) = \eta P(z_t \mid X_t)P(X_t \mid z_{1:t-1}).$$

This is exactly the two-step filter we just used.

Key idea

The HMM forward update compresses an arbitrarily long observation history into the current belief vector.

Why This Avoids the History Explosion

Suppose the HMM has n discrete states.

We do *not* retain all n^{T+1} trajectories.

We retain only

$$bel_t = [P(X_t = s_1 \mid z_{1:t}), \dots, P(X_t = s_n \mid z_{1:t})].$$

A dense transition update costs roughly

$$O(n^2)$$

per time step rather than enumerating all histories.

Key idea

Dynamic programming works because many different histories lead to the same current state and can be merged into one probability mass.

HMM Tradeoff: The State Must Be Good Enough

The Markov assumption is only useful if X_t actually captures the information needed for future prediction.

Bad state choice:

- ▶ omit velocity, and position may no longer predict next position well;
- ▶ omit sensor health, and observation errors may look inexplicable;
- ▶ omit crowd flow, and motion uncertainty may become strongly history-dependent.

Key idea

The price of Markov tractability is a state representation rich enough to make the future approximately conditionally independent of the distant past.

HMMs Are Excellent When the Hidden State Is Discrete

Examples:

- ▶ robot is in one of a finite set of rooms;
- ▶ machine is in one of several health modes;
- ▶ user is in one of several activity states;
- ▶ speech system is in one of a sequence of phonetic states.

But our delivery robot's pose is not naturally

$$\{A, B, C\}.$$

It is closer to

$$(x, y, \theta) \in \mathbb{R}^2 \times [-\pi, \pi).$$

Key idea

The next question is how to maintain a belief over a continuous state without discretizing the world into millions of bins.

Why Not Just Discretize the Continuous Pose?

Suppose campus localization uses:

- ▶ 1000 possible x bins;
- ▶ 1000 possible y bins;
- ▶ 360 heading bins.

Then the pose grid already contains

$$1000 \times 1000 \times 360 = 360,000,000$$

discrete states.

Before adding velocity, sensor bias, or anything else.

Key idea

Fine discretization can recover continuous detail, but the state count explodes combinatorially.

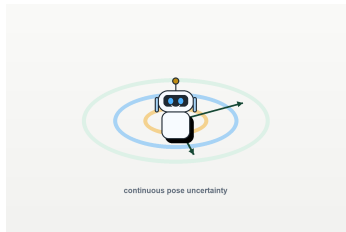
What If the Belief Has a Simple Shape?

Instead of storing every possible continuous state, suppose the belief is Gaussian:

$$X_t \sim \mathcal{N}(\mu_t, \Sigma_t).$$

Then the full belief is summarized by:

- ▶ a mean vector μ_t ;
- ▶ a covariance matrix Σ_t .



Key idea

A parametric filter gains efficiency by assuming the belief can be represented with a small number of parameters.

The Linear-Gaussian State-Space Model

Kalman filtering assumes linear dynamics with Gaussian noise:

$$X_t = AX_{t-1} + BU_t + \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, Q).$$

Measurements are also linear with Gaussian noise:

$$Z_t = HX_t + \delta_t, \quad \delta_t \sim \mathcal{N}(0, R).$$

Key idea

Linear transformations of Gaussian variables remain Gaussian, so prediction and correction stay inside the same compact family.

Kalman Filtering Is Still Predict Then Correct

Predict:

$$\mu_t^- = A\mu_{t-1} + BU_t$$

$$\Sigma_t^- = A\Sigma_{t-1}A^T + Q$$

Correct:

$$K_t = \Sigma_t^- H^T (H\Sigma_t^- H^T + R)^{-1}$$

$$\mu_t = \mu_t^- + K_t(Z_t - H\mu_t^-)$$

Key idea

The Kalman gain decides how strongly to trust the new measurement relative to the predicted uncertainty.

One-Dimensional Kalman Example

Suppose before the measurement:

$$X_t \sim \mathcal{N}(12, 5).$$

The position sensor reports

$$z_t = 11, \quad R = 4.$$

Kalman gain:

$$K = \frac{5}{5 + 4} = 0.556.$$

Updated mean:

$$\mu_t = 12 + 0.556(11 - 12) \approx 11.44.$$

Updated variance:

$$\Sigma_t = (1 - K)5 \approx 2.22.$$

Key idea

The estimate moves toward the measurement, while uncertainty decreases because prediction and sensing provide complementary information.

Why Kalman Filters Are So Attractive

- ▶ continuous state without massive discretization;
- ▶ compact belief representation;
- ▶ recursive online updates;
- ▶ mathematically exact for the linear-Gaussian model;
- ▶ efficient matrix operations;
- ▶ natural fusion of multiple noisy measurements.

Key idea

Strong assumptions buy an unusually efficient and elegant state estimator.

But the Gaussian Assumption Can Be the Wrong Shape

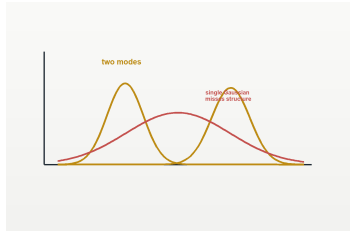
Imagine two visually identical campus corridors.

The robot may plausibly be in either one.

The belief is now **multimodal**:

$$p(x) \approx \text{peak near corridor 1} + \text{peak near corridor 2}.$$

One Gaussian tends to summarize that as a mean location *between* the two possibilities.



Key idea

A compact parameterization can be computationally excellent while representing the wrong uncertainty geometry.

Kalman Failure Modes Motivate a Different Representation

Standard Kalman filtering assumes:

- ▶ linear transition model;
- ▶ linear observation model;
- ▶ Gaussian process noise;
- ▶ Gaussian sensor noise;
- ▶ effectively unimodal belief.

Real localization can be nonlinear, asymmetric, and multimodal.

Key idea

If the posterior cannot be represented well by one Gaussian, we need a belief representation that can take almost any shape.

Lecture 20 Comes Back: Represent the Belief with Samples

A particle filter approximates the current belief with weighted samples:

$$bel_t(x) \approx \{(x_t^{[1]}, w_t^{[1]}), \dots, (x_t^{[M]}, w_t^{[M]})\}.$$

Each particle is one hypothesis about the current state.

Many particles in one region means high probability mass there.



particle hypotheses cluster in plausible locations

Key idea

Particle filtering replaces one analytic distribution with a population of explicit state hypotheses.

Particle Filtering: Predict, Weight, Resample

At each time step:

1. **Predict:** sample each particle through the motion model.
2. **Weight:** score how well each predicted particle explains the observation.
3. **Normalize:** convert scores into a probability distribution.
4. **Resample:** preferentially copy high-weight particles and discard low-weight ones.

$$w_t^{[i]} \propto P(z_t \mid x_t^{[i]}).$$

Key idea

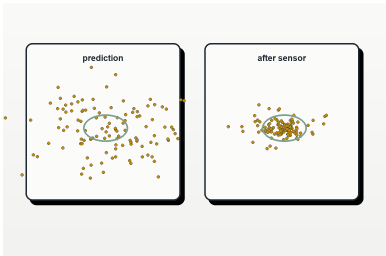
Particle filtering is sequential importance sampling plus resampling, applied repeatedly as evidence arrives.

Why Resampling Matters

Without resampling, many particles eventually carry almost no weight.

After resampling:

- ▶ hypotheses consistent with the sensor are duplicated;
- ▶ inconsistent hypotheses disappear;
- ▶ computation is concentrated where posterior probability is high.



Key idea

Resampling reallocates a fixed computational budget toward the parts of state space that currently matter.

Particle Filters Can Represent Multiple Hypotheses

Suppose the robot could be in either of two similar corridors.

A particle set can maintain two clusters simultaneously.

New sensor evidence may later eliminate one cluster.

Kalman belief

One Gaussian center + covariance.

Particle belief

Arbitrary shape, including multiple separated modes.

Key idea

Particle filters trade analytic compactness for representational flexibility.

Particle Filters Have Their Own Failure Modes

- ▶ too few particles can miss important regions entirely;
- ▶ high-dimensional states may require very many particles;
- ▶ repeated resampling can reduce diversity;
- ▶ poor motion or sensor models still produce poor estimates;
- ▶ computation is stochastic rather than deterministic.

Key idea

Sampling avoids some analytic assumptions, but it pays with Monte Carlo variance and potentially large particle counts.

AMCL: Adaptive Monte Carlo Localization

In ROS 2 Navigation, AMCL is a particle-filter-based localizer for estimating robot pose in a known static map.

It combines:

- ▶ a prior / current cloud of pose particles;
- ▶ odometry-based motion updates;
- ▶ laser-scan likelihoods against the map;
- ▶ resampling to concentrate hypotheses;
- ▶ an estimated pose distribution for navigation.

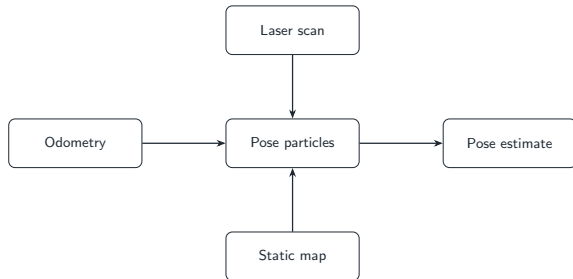
Key idea

AMCL is not just an abstract probability example: it operationalizes particle filtering for practical mobile-robot localization.

What AMCL Is Estimating

The hidden state is approximately robot pose:

$$X_t = (x_t, y_t, \theta_t).$$



Evidence comes from a range sensor scored against a known map; motion information comes from odometry.

Key idea

Localization is filtering: estimate current pose from motion history and sensor evidence without storing every past trajectory.

One Problem, Three Belief Representations

HMM

State: discrete

Belief: probability vector

Exact? yes, under the model

Multimodal? yes, discrete

Cost: state count

Assumption: Markov + finite state

Kalman

State: continuous

Belief: Gaussian (μ, Σ)

Exact? linear-Gaussian

Multimodal? poorly

Cost: matrix algebra

Assumption: linear + Gaussian

Particle filter

State: general / continuous

Belief: weighted samples

Exact? approximate

Multimodal? yes

Cost: particle count

Assumption: representative samples

Key idea

The filtering idea is shared; the algorithms differ mainly in how they represent and update the belief distribution.

The Whole Lecture in One Flow

1. Static Bayesian network: reason about one snapshot.
2. Dynamic Bayesian network: repeat structure through time.
3. Filtering: maintain $P(X_t \mid z_{1:t})$ online.
4. Markov assumption: summarize relevant history in the current state belief.
5. Choose a belief representation appropriate to the problem.

HMM

Discrete state
Probability vector

Kalman

Continuous state
Gaussian belief

Particle filter

General state
Weighted samples

Key idea

State estimation is a sequence of tradeoffs: what state to remember, what uncertainty shape to assume, and how much computation to spend.

Takeaways

- ▶ Dynamic Bayesian networks extend probabilistic graphical models across time.
- ▶ Filtering recursively combines a transition model with new evidence.
- ▶ Explicit state histories explode; the Markov assumption lets the current belief summarize the relevant past.
- ▶ HMMs provide exact filtering for compact discrete hidden states.
- ▶ Kalman filters compress continuous linear-Gaussian beliefs into a mean and covariance.
- ▶ Particle filters approximate flexible, multimodal beliefs with weighted samples.
- ▶ AMCL applies particle filtering to practical map-based mobile-robot localization.

Key idea

The central engineering question is not “which filter is best?” It is “which representation of uncertainty matches the structure of this state-estimation problem?”